

Лабораторная работа № 7

Централизованное резервное копирование и восстановление с помощью BorgBackup

Продолжительность

2 занятия по 90 минут.

Среда выполнения

- VirtualBox;
 - Ubuntu Server или Debian Server;
 - сервер-источник web01;
 - сервер резервного копирования backup01;
 - BorgBackup 1.x;
 - OpenSSH Server.
-

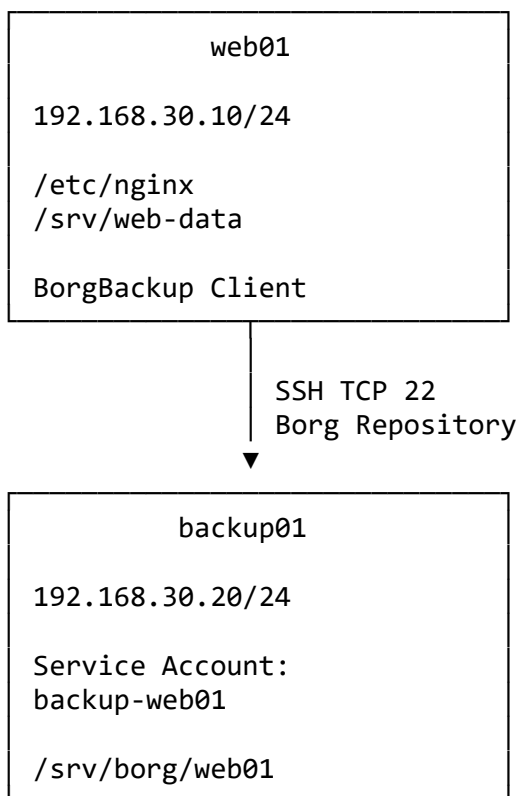
1. Цель работы

Настроить централизованное резервное копирование данных Linux-сервера и доказать возможность их восстановления.

В ходе работы необходимо:

- подготовить сервер резервного копирования;
 - создать отдельную сервисную учётную запись;
 - настроить аутентификацию по SSH-ключу;
 - ограничить назначение SSH-ключа;
 - создать зашифрованный Borg-репозиторий;
 - выполнить минимум два резервных копирования;
 - проверить дедупликацию и список архивов;
 - применить политику хранения;
 - проверить целостность репозитория;
 - восстановить данные в отдельный каталог;
 - выполнить прикладную проверку восстановленных файлов;
 - измерить фактическое время восстановления.
-

2. Схема стенда



3. Таблица адресации

Устройство	IP-адрес	Маска	Назначение
web01	192.168.30.10	255.255.255.0	Источник данных
backup01	192.168.30.20	255.255.255.0	Хранение резервных копий

Оба сервера подключаются к одной сети VirtualBox:

Internal Network: SERVER-LAN

4. Архитектура резервного копирования

В лаборатории используется Push-модель:

web01 → backup01

web01 самостоятельно:

1. читает локальные файлы;

2. подключается к backup01;
3. создаёт архив Borg;
4. отправляет новые блоки в удалённый репозиторий.

На backup01 используется отдельная учётная запись:

backup-web01

Она не должна иметь доступа к резервным копиям других серверов.

5. Исходные данные

В резервную копию включаются:

```
/etc/nginx  
/srv/web-data
```

В архив не включаются:

- временные файлы;
 - кэш;
 - PCAP;
 - дампы, которые создаются другим процессом;
 - каталоги виртуальных машин;
 - содержимое /proc, /sys, /dev и /run.
-

6. Базовые задания

Задание 1. Проверить исходный стенд

На web01 смотрим имя, адрес и маршрут:

```
hostnamectl  
ip -br addr  
ip route
```

Проверяем доступность backup01:

```
ping -c 4 192.168.30.20
```

На backup01:

```
hostnamectl  
ip -br addr  
ip route
```

Проверяем доступность web01:

```
ping -c 4 192.168.30.10
```

На обоих серверах проверить время:

```
date  
timedatectl
```

Ожидаемый результат

- серверы доступны друг другу;
- адреса соответствуют таблице;
- время на серверах различается незначительно;
- маршрут между серверами существует.

Точное время необходимо для правильного определения возраста резервной копии и анализа журналов.

Задание 2. Подготовить данные на web01

Установить Nginx и дополнительные программы:

```
sudo apt update  
sudo apt install -y nginx curl python3
```

Создать каталог данных:

```
sudo mkdir -p /srv/web-data
```

Создать первую версию веб-страницы:

```
sudo tee /srv/web-data/index.html >/dev/null <<'EOF'  
<!doctype html>  
<html lang="ru">  
<head>  
  <meta charset="utf-8">  
  <title>web01</title>  
</head>  
<body>  
  <h1>web01 – версия 1</h1>  
  <p>Этот файл должен попасть в первый архив Borg.</p>  
</body>  
</html>  
EOF
```

Создать дополнительный файл:

```
sudo tee /srv/web-data/important.txt >/dev/null <<'EOF'  
Важные данные приложения web01.  
Версия: 1  
EOF
```

Настроить Nginx на использование /srv/web-data.

Создать резервную копию конфигурации:

```
sudo cp -a \  
  /etc/nginx/sites-available/default \  
  /etc/nginx/sites-available/default.before-lab7
```

Открыть файл:

```
sudo nano /etc/nginx/sites-available/default
```

Найти:

```
root /var/www/html;
```

Заменить:

```
root /srv/web-data;
```

Проверить синтаксис:

```
sudo nginx -t
```

Применить:

```
sudo systemctl reload nginx
```

Проверить сайт:

```
curl http://127.0.0.1/
```

Проверить файлы:

```
sudo find /srv/web-data \  
  -maxdepth 1 \  
  -type f \  
  -printf '%f %s bytes\n'
```

Создать контрольные суммы:

```
sudo sha256sum \  
  /srv/web-data/index.html \  
  /srv/web-data/important.txt \  
  | sudo tee /root/web-data-v1.sha256
```

Задание 3. Установить BorgBackup и SSH

На backup01:

```
sudo apt update  
sudo apt install -y borgbackup openssh-server
```

Включить SSH:

```
sudo systemctl enable --now ssh
```

Проверить:

```
systemctl is-active ssh  
sudo ss -lntp | grep ':22'  
borg --version
```

На web01:

```
sudo apt update  
sudo apt install -y borgbackup openssh-client
```

Проверить:

```
borg --version  
ssh -V
```

Важно

Примеры лабораторной работы ориентированы на синтаксис BorgBackup 1.x.

Фактическую версию необходимо записать в отчёт:

```
borg --version
```

Задание 4. Создать сервисную учётную запись на backup01

На backup01 создать пользователя:

```
sudo adduser \  
  --disabled-password \  
  --gecos '' \  
  backup-web01
```

Проверить:

```
id backup-web01  
getent passwd backup-web01
```

Создать каталог репозитория:

```
sudo mkdir -p /srv/borg/web01
```

Назначить владельца:

```
sudo chown \  
  backup-web01:backup-web01 \  
  /srv/borg/web01
```

Ограничить доступ:

```
sudo chmod 700 /srv/borg/web01
```

Проверить:

```
sudo ls -ld /srv/borg/web01
```

Ожидаемый результат:

```
drwx----- ... backup-web01 backup-web01 /srv/borg/web01
```

Почему используется отдельная учётная запись

Личная учётная запись администратора не должна использоваться для автоматического резервного копирования.

Компрометация ключа web01 должна затрагивать только репозиторий этого сервера.

Задание 5. Настроить ограниченный SSH-ключ

На web01 перейти в root-сессию:

```
sudo -i
```

Создать каталог SSH:

```
install -d -m 700 /root/.ssh
```

Создать отдельный ключ Borg:

```
ssh-keygen \  
-t ed25519 \  
-f /root/.ssh/id_ed25519_borg \  
-N '' \  
-C 'web01 Borg backup'
```

Проверить:

```
ls -l /root/.ssh/id_ed25519_borg*
```

Ожидаемые права:

```
-rw----- id_ed25519_borg  
-rw-r--r-- id_ed25519_borg.pub
```

Показать public key:

```
cat /root/.ssh/id_ed25519_borg.pub
```

Закрытый файл:

```
/root/.ssh/id_ed25519_borg
```

нельзя копировать на backup01, отправлять в отчёт или публиковать.

Проверка SSH Host Key

На консоли backup01 получить fingerprint:

```
sudo ssh-keygen \  
-lf /etc/ssh/ssh_host_ed25519_key.pub
```

На web01 получить ключ сервера:

```
ssh-keyscan \  
-t ed25519 \  
192.168.30.20 \  
> /tmp/backup01-ed25519.pub
```

Посмотреть fingerprint:

```
ssh-keygen \  
-lf /tmp/backup01-ed25519.pub
```

Сравнить fingerprint с результатом на backup01.

Только после совпадения добавить Host Key:

```
cat /tmp/backup01-ed25519.pub \  
>> /root/.ssh/known_hosts
```

Установить права:

```
chmod 600 /root/.ssh/known_hosts
```

Установка public key на backup01

На backup01 создать SSH-каталог:

```
sudo install \  
-d \  
-m 700 \  
-o backup-web01 \  
-g backup-web01 \  
/home/backup-web01/.ssh
```

Открыть файл:

```
sudo nano /home/backup-web01/.ssh/authorized_keys
```

Добавить одной строкой:

```
command="/usr/bin/borg serve --restrict-to-path /srv/borg/web01",restrict ssh-  
ed25519 ПУБЛИЧНЫЙ_КЛЮЧ web01 Borg backup
```

Вместо:

```
ssh-ed25519 ПУБЛИЧНЫЙ_КЛЮЧ web01 Borg backup
```


вставить содержимое файла:

```
/root/.ssh/id_ed25519_borg.pub
```

с сервера web01.

Назначить владельца и права:

```
sudo chown \
  backup-web01:backup-web01 \
  /home/backup-web01/.ssh/authorized_keys
```

```
sudo chmod 600 \
  /home/backup-web01/.ssh/authorized_keys
```

Проверить:

```
sudo ls -ld /home/backup-web01/.ssh
sudo ls -l /home/backup-web01/.ssh/authorized_keys
```

Ограничения ключа

Данный ключ:

- не создаёт интерактивную shell-сессию;
 - не разрешает PTY;
 - не разрешает port forwarding;
 - запускает только borg serve;
 - ограничен каталогом /srv/borg/web01.
-

Задание 6. Создать зашифрованный Borg-репозиторий

Все следующие команды на web01 выполняются в root-сессии.

Создать каталог конфигурации:

```
install -d -m 700 /root/.config/borg
```

Создать файл с парольной фразой:

```
nano /root/.config/borg/passphrase
```

Ввести собственную учебную парольную фразу.

Фраза не должна совпадать с паролем пользователя.

Установить права:

```
chmod 600 /root/.config/borg/passphrase
```

Парольную фразу запрещено:

- добавлять в Git;

- вставлять в отчёт;
- помещать прямо в backup-скрипт;
- отправлять в общий чат.

Настроить переменные текущей root-сессии:

```
export BORG_REPO='ssh://backup-web01@192.168.30.20/srv/borg/web01'
```

```
export BORG_RSH='ssh -i /root/.ssh/id_ed25519_borg'
```

```
export BORG_PASSCOMMAND='cat /root/.config/borg/passphrase'
```

Проверить адрес репозитория:

```
printf 'BORG_REPO=%s\n' "$BORG_REPO"
```

Инициализировать репозиторий:

```
borg init \  
  --encryption=repokey-blake2 \  
  "$BORG_REPO"
```

Проверить:

```
borg info "$BORG_REPO"
```

Ожидаемый результат

Borg выводит информацию о репозитории без ошибок аутентификации.

Репозиторий пока не содержит архивов.

Важно

При шифровании repokey для восстановления нужны:

- сам репозиторий;
- парольная фраза;
- доступ по SSH;
- совместимая версия Borg.

Потеря парольной фразы делает резервную копию недоступной.

Задание 7. Создать первый архив

Перед резервным копированием проверить источники:

```
test -d /etc/nginx  
test -f /srv/web-data/index.html  
test -f /srv/web-data/important.txt
```

Проверить размер:

```
du -sh /etc/nginx /srv/web-data
```

Создать первый архив:

```
borg create \
  --stats \
  --show-rc \
  --compression lz4 \
  ':::{hostname}-{now:%Y-%m-%d_%H-%M-%S}' \
  /etc/nginx \
  /srv/web-data
```

Посмотреть список архивов:

```
borg list
```

Посмотреть последний архив:

```
borg list \
  --last 1 \
  --format '{archive} {time} {size}{NL}'
```

Посмотреть содержимое:

```
borg list \
  --last 1 \
  | grep -E 'etc/nginx|srv/web-data'
```

Проверить информацию о репозитории:

```
borg info "$BORG_REPO"
```

Ожидаемый результат

В репозитории появился один архив с именем вида:

```
web01-2026-07-28_15-30-00
```

Точная дата зависит от времени выполнения работы.

Задание 8. Изменить данные и создать второй архив

Изменить веб-страницу:

```
sudo tee /srv/web-data/index.html >/dev/null <<'EOF'
<!doctype html>
<html lang="ru">
<head>
  <meta charset="utf-8">
  <title>web01</title>
</head>
<body>
  <h1>web01 – версия 2</h1>
```

```
<p>Этот файл изменён после первого резервного копирования.</p>
</body>
</html>
EOF
```

Изменить текстовый файл:

```
sudo tee -a /srv/web-data/important.txt >/dev/null <<'EOF'
Версия: 2
Добавлена новая строка после первого backup.
EOF
```

Создать новый файл:

```
sudo tee /srv/web-data/new-file.txt >/dev/null <<'EOF'
Файл создан перед вторым архивом Borg.
EOF
```

Проверить сайт:

```
curl http://127.0.0.1/
```

Создать новые контрольные суммы:

```
sudo sha256sum \
  /srv/web-data/index.html \
  /srv/web-data/important.txt \
  /srv/web-data/new-file.txt \
  | sudo tee /root/web-data-v2.sha256
```

Создать второй архив:

```
borg create \
  --stats \
  --show-rc \
  --compression lz4 \
  '::{hostname}-{now:%Y-%m-%d_%H-%M-%S}' \
  /etc/nginx \
  /srv/web-data
```

Проверить список:

```
borg list \
  --format '{archive} {time}{NL}'
```

Проверить статистику:

```
borg info "$BORG_REPO"
```

Наблюдение

Второй архив логически содержит полный набор файлов.

Физически одинаковые блоки не записываются повторно благодаря дедупликации Borg.

Задание 9. Проверить retention и целостность

Посмотреть, какие архивы затронет политика хранения:

```
borg prune \  
  --dry-run \  
  --list \  
  --keep-daily=7 \  
  --keep-weekly=4 \  
  --keep-monthly=6 \  
  "$BORG_REPO"
```

Поскольку создано только два свежих архива, удаление обычно не выполняется.

Применить политику:

```
borg prune \  
  --list \  
  --keep-daily=7 \  
  --keep-weekly=4 \  
  --keep-monthly=6 \  
  "$BORG_REPO"
```

Проверить список:

```
borg list
```

Выполнить проверку репозитория:

```
borg check \  
  --verify-data \  
  "$BORG_REPO"
```

Проверить код завершения:

```
rc=$?  
echo "borg_check_exit_code=$rc"
```

Ожидаемый код:

0

Важно

`borg check` подтверждает читаемость и структуру репозитория.

Он не доказывает, что:

- в архив попали нужные файлы;
 - приложение сможет запуститься;
 - восстановление укладывается в RTO.
-

Задание 10. Выполнить неразрушающее восстановление

Посмотреть имена архивов:

```
borg list --short
```

Сохранить имя самого старого архива:

```
OLD_ARCHIVE=$(  
  borg list \  
    --short \  
    --first 1  
)
```

Сохранить имя последнего архива:

```
LATEST_ARCHIVE=$(  
  borg list \  
    --short \  
    --last 1  
)
```

Проверить:

```
printf 'OLD=%s\n' "$OLD_ARCHIVE"  
printf 'LATEST=%s\n' "$LATEST_ARCHIVE"
```

Создать каталоги восстановления:

```
rm -rf /restore-test/web01  
mkdir -p \  
  /restore-test/web01/version1 \  
  /restore-test/web01/version2
```

Восстановление первой версии

Зафиксировать время начала:

```
RESTORE_START=$(date +%s)
```

Перейти в каталог:

```
cd /restore-test/web01/version1
```

Извлечь выбранные данные:

```
borg extract \  
  "::${OLD_ARCHIVE}" \  
  etc/nginx \  
  srv/web-data
```

Восстановление второй версии

Перейти в другой каталог:

```
cd /restore-test/web01/version2
```

Извлечь:

```
borg extract \
  "::$LATEST_ARCHIVE" \
  etc/nginx \
  srv/web-data
```

Зафиксировать окончание:

```
RESTORE_END=$(date +%s)
```

Рассчитать время:

```
RESTORE_SECONDS=$((RESTORE_END - RESTORE_START))
printf 'restore_time_seconds=%s\n' "$RESTORE_SECONDS"
```

Проверка восстановленных данных

Посмотреть структуру:

```
find /restore-test/web01 \
  -maxdepth 5 \
  -type f \
  -printf '%p %s bytes\n'
```

Проверить первую версию:

```
cat \
  /restore-test/web01/version1/srv/web-data/index.html
```

Проверить вторую версию:

```
cat \
  /restore-test/web01/version2/srv/web-data/index.html
```

Убедиться, что нового файла нет в первой версии:

```
test ! -e \
  /restore-test/web01/version1/srv/web-data/new-file.txt
```

Убедиться, что файл есть во второй:

```
test -f \
  /restore-test/web01/version2/srv/web-data/new-file.txt
```

Сравнить версии:

```
diff -ru \  
  /restore-test/web01/version1/srv/web-data \  
  /restore-test/web01/version2/srv/web-data
```

Прикладная проверка

Запустить временный HTTP-сервер из восстановленного каталога:

```
python3 \  
  -m http.server 8080 \  
  --directory /restore-test/web01/version2/srv/web-data \  
  >/tmp/restore-http.log 2>&1 &
```

Сохранить PID:

```
RESTORE_HTTP_PID=$!
```

Проверить восстановленный сайт:

```
curl http://127.0.0.1:8080/
```

Остановить тестовый сервер:

```
kill "$RESTORE_HTTP_PID"
```

Проверить, что процесс завершён:

```
wait "$RESTORE_HTTP_PID" 2>/dev/null || true
```

Ожидаемый результат

- данные восстановлены не поверх рабочей системы;
 - первая версия отличается от второй;
 - восстановленный сайт доступен через тестовый HTTP-сервер;
 - фактическое время восстановления зафиксировано.
-

7. Проверка RPO и RTO

RPO

Посмотреть дату последнего архива:

```
borg list \  
  --last 1 \  
  --format '{time} {archive}{NL}'
```

RPO оценивается от последней подтверждённой успешной резервной копии, а не от записи в расписании.

Пример:

Требуемый RPO: 24 часа.

Возраст последнего проверенного архива: 15 минут.

Требование выполняется.

RTO

Зафиксировать:

- время обнаружения проблемы;
- время выбора архива;
- время извлечения;
- время проверки файлов;
- время запуска тестового сервиса.

Команда `borg extract` может работать несколько секунд, но это ещё не полный RTO.

8. Дополнительные задания

Дополнительное задание 1. Сравнить BorgBackup и rsync

На `backup01` создать каталог `mirror`:

```
sudo mkdir -p /srv/rsync/web01/current
sudo chown \
  backup-web01:backup-web01 \
  /srv/rsync/web01/current
```

Временно разрешить обычную SSH-сессию отдельным учебным ключом либо выполнить задание через локальный каталог, предоставленный преподавателем.

На `web01` сначала выполнить `dry-run`:

```
sudo rsync \
  -aHAXn \
  /srv/web-data/ \
  backup-web01@192.168.30.20:/srv/rsync/web01/current/
```

Обратить внимание на завершающий `/`:

```
/srv/web-data/
```

Он означает копирование содержимого каталога.

После проверки убрать `-n`:

```
sudo rsync \
  -aHAX \
```

```
/srv/web-data/ \
backup-web01@192.168.30.20:/srv/rsync/web01/current/
```

Удалить на источнике тестовый файл:

```
sudo rm /srv/web-data/new-file.txt
```

Повторить dry-run с --delete:

```
sudo rsync \
-aHAXn \
--delete \
/srv/web-data/ \
backup-web01@192.168.30.20:/srv/rsync/web01/current/
```

Вывод

Обычный rsync mirror хранит текущее состояние.

При использовании --delete удаление файла на источнике может повториться на приёмнике.

Borg хранит версионные архивы и позволяет восстановить старое состояние.

Дополнительное задание 2. Автоматизировать backup через systemd timer

На web01 создать файл окружения:

```
sudo install -d -m 700 /etc/borg
```

```
sudo nano /etc/borg/web01.env
```

Добавить:

```
BORG_REPO=ssh://backup-web01@192.168.30.20/srv/borg/web01
BORG_RSH=ssh -i /root/.ssh/id_ed25519_borg
BORG_PASSCOMMAND=cat /root/.config/borg/passphrase
```

Ограничить права:

```
sudo chmod 600 /etc/borg/web01.env
```

Создать сценарий:

```
sudo nano /usr/local/sbin/web01-backup.sh
```

Добавить:

```
#!/usr/bin/env bash
```

```
set -euo pipefail
```

```
PATH='/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin'
export PATH
```

```
exec 9>/run/lock/web01-backup.lock
```

```
if ! flock -n 9; then
    logger -p user.warning -t web01-backup \
        'Другой экземпляр backup уже работает'
    exit 1
fi

if [[ ! -f /srv/web-data/index.html ]]; then
    logger -p user.err -t web01-backup \
        'Источник /srv/web-data не содержит index.html'
    exit 1
fi
```

```
borg create \
    --stats \
    --show-rc \
    --compression lz4 \
    '::{hostname}-{now:%Y-%m-%d_%H-%M-%S}' \
    /etc/nginx \
    /srv/web-data
```

```
borg prune \
    --list \
    --keep-daily=7 \
    --keep-weekly=4 \
    --keep-monthly=6 \
    "$BORG_REPO"
```

```
logger -t web01-backup \
    'Резервное копирование завершено'
```

Назначить права:

```
sudo chmod 750 /usr/local/sbin/web01-backup.sh
```

Проверить синтаксис:

```
sudo bash -n /usr/local/sbin/web01-backup.sh
```

Создать service:

```
sudo nano /etc/systemd/system/web01-backup.service
```

Добавить:

```
[Unit]
Description=Borg backup of web01
After=network-online.target
Wants=network-online.target
```

```
[Service]
Type=oneshot
EnvironmentFile=/etc/borg/web01.env
ExecStart=/usr/local/sbin/web01-backup.sh
Nice=10
IOSchedulingClass=best-effort
```

Создать timer:

```
sudo nano /etc/systemd/system/web01-backup.timer
```

Добавить:

```
[Unit]
Description=Daily Borg backup of web01
```

```
[Timer]
OnCalendar=daily
Persistent=true
RandomizedDelaySec=10m
```

```
[Install]
WantedBy=timers.target
```

Применить:

```
sudo systemctl daemon-reload
sudo systemctl enable --now web01-backup.timer
```

Запустить вручную:

```
sudo systemctl start web01-backup.service
```

Проверить:

```
systemctl status web01-backup.service --no-pager
systemctl list-timers web01-backup.timer
journalctl -u web01-backup.service --no-pager
```

Дополнительное задание 3. Проверить ошибку источника

Переименовать каталог данных:

```
sudo mv \
  /srv/web-data \
  /srv/web-data.offline
```

Запустить автоматический backup:

```
sudo systemctl start web01-backup.service
```

Проверить:

```
systemctl status web01-backup.service --no-pager
journalctl \
  -u web01-backup.service \
  -n 30 \
  --no-pager
```

Ожидаемый результат

Задание завершается ошибкой до создания архива, потому что ожидаемый источник отсутствует.

Вернуть каталог:

```
sudo mv \
  /srv/web-data.offline \
  /srv/web-data
```

Повторить:

```
sudo systemctl start web01-backup.service
```

Проверить новый архив:

```
sudo -i
source /etc/borg/web01.env
borg list --last 3
```

Вывод

Exit code 0 бесполезен, если сценарий успешно скопировал пустой или неправильный каталог.

До резервного копирования нужно проверять наличие ожидаемых данных и точек монтирования.

9. Основные команды BorgBackup

Репозиторий

```
borg init --encryption=repokey-blake2 "$BORG_REPO"
borg info "$BORG_REPO"
```

Создание архива

```
borg create \
  --stats \
  '::{hostname}-{now:%Y-%m-%d_%H-%M-%S}' \
  /etc/nginx \
  /srv/web-data
```

Просмотр

```
borg list
borg list "::archive-name"
borg info "::archive-name"
```

Проверка

```
borg check --verify-data "$BORG_REPO"
```

Retention

```
borg prune \
  --dry-run \
  --list \
  --keep-daily=7 \
  --keep-weekly=4 \
  --keep-monthly=6 \
  "$BORG_REPO"
```

Восстановление

```
mkdir -p /restore-test/web01
cd /restore-test/web01
borg extract "::archive-name"
```

10. Требования к отчёту

В отчёте представить:

1. Название и цель работы.
2. Схему стенда и таблицу адресации.
3. Версию BorgBackup.
4. Название сервисной учётной записи.
5. Применённые ограничения SSH-ключа.
6. Адрес Borg-репозитория.
7. Список минимум из двух архивов.
8. Результат `borg check`.
9. Политику retention.
10. Содержимое восстановленных версий `index.html`.
11. Результат прикладной проверки через HTTP.
12. Фактическое время восстановления.
13. Оценку RPO и RTO.
14. Краткий вывод по работе.

В отчёт запрещено добавлять:

- приватный SSH-ключ;
 - парольную фразу Borg;
 - экспортированный ключ Borg;
 - полное содержимое `authorized_keys`;
 - реальные учётные данные.
-

11. Чек-лист выполнения

- ☐ `web01` достигает `backup01`.
 - ☐ На обоих серверах установлена совместимая версия Borg.
 - ☐ На `backup01` создан пользователь `backup-web01`.
 - ☐ Репозиторий принадлежит сервисной учётной записи.
 - ☐ SSH Host Key проверен по `fingerprint`.
 - ☐ Создан отдельный SSH-ключ Borg.
 - ☐ Закрытый ключ имеет права `600`.
 - ☐ Public key ограничен командой `borg serve`.
 - ☐ Borg-репозиторий зашифрован.
 - ☐ Парольная фраза не сохранена в Git.
 - ☐ Создано минимум два архива.
 - ☐ Вторая версия данных отличается от первой.
 - ☐ Выполнен `borg check`.
 - ☐ Retention сначала проверен через `dry-run`.
 - ☐ Restore выполнен в `/restore-test`.
 - ☐ Рабочие данные не перезаписывались.
 - ☐ Проверены обе восстановленные версии.
 - ☐ Восстановленный сайт прошёл HTTP-проверку.
 - ☐ Измерено фактическое время восстановления.
 - ☐ Оценены RPO и RTO.
-

12. Контрольные вопросы

1. Почему RAID, snapshot и replication не заменяют резервное копирование?
2. Чем Push-модель отличается от Pull-модели?
3. Почему для backup используется отдельная сервисная учётная запись?
4. Зачем ограничивать SSH-ключ командой `borg serve`?
5. Почему успешный exit code не доказывает полноту резервной копии?
6. Чем проверка целостности репозитория отличается от Test Restore?

7. Почему восстановление выполняется в отдельный каталог?
8. Чем RPO отличается от RTO?